



Anil Seth

Using CPU Cores with Erlang

The author tells you why Erlang is an excellent option to write your own code as he delves deeper into the world of this programming language.

A few months ago, an article in *OSFY* on Erlang reminded me that I had never got around to exploring it. Erlang is fascinating because, apart from its other features, it was designed to:

- Avoid catastrophic failure
- Be able to scale to a system with a massive number of processes
- Communicate using asynchronous messages between processes

These features make it natural for Erlang to use multiple cores of a processor. It also makes it easy to develop a distributed system.

You can get started with tutorials on the erlang.org site. Learning the syntax of a new language is not hard. Since it is a functional programming language, you may face a couple of challenges when moving from an imperative or object-oriented language.

- You need to forget about loops and iteration and think recursion. However, in Erlang, functions like *foreach* and *map* make it easy, especially if you are already familiar with *map* in Python.
- You need to forget about variables, storing results in them and manipulating them. For me, this was the harder task.

Concurrent programming

Even if you have not done the tutorials, you should not find it hard to read Erlang code. So, consider a function, *n_echo*, which echoes a message 'N' times. Write the following code in a text file, *cp.erl*:

```
% Module demos concurrent processes
-module(cp).
-export([n_echo/2]).
n_echo(Msg, 0) ->
done;
n_echo(Msg, N) ->
io:format("~s~n", [Msg]),
n_echo(Msg, N-1).
```

The syntax of the format function is similar to the print function in Python 3, with ~ replacing \.

In the Erlang shell (erl command), type the following:

```
1> c(cp).
2> cp:n_echo(one, 3).
3> cp:n_echo(two, 3).
```

As you would expect, you get *one* printed three times, followed by *two* printed three times. In order to run these functions concurrently, you can spawn processes. Define a convenience function, *Spawn_echo*:

```
4> spawn_echo = fun(Msg) ->
5> spawn(cp, n_echo, [Msg,3])
6> end.
```

The parameters of *spawn* are the module name, the function name and the list of parameters. Now, you can use this convenience function to spawn a list of processes:

```
7> lists:foreach(Spawn_echo, [one, two, three]).
```

The *foreach* function will apply the *Spawn_echo* function to each element of the list. Now, you will find that *one*, *two* and *three* are printed three times each but are running concurrently. Programming concurrency in Erlang is about as easy as it can be.

Now, consider the example of two programs, *ping* and *pong*. *Ping* sends a message to *pong* and waits for a reply. *Pong* is waiting for a message and responds after receiving a message. Add these methods to the *cp.erl* file and modify the export line as follows:

```
-export([n_echo/2, ping/3, pong/0, ping_pong/0]).
ping(0, Pong_PID, Ping) ->
Pong_PID ! quit,
io:format("ping finished~n", []);
ping(N, Pong_PID, Ping) ->
Pong_PID! {Ping, self()},
receive
X ->
io:format("Ping received ~w~n", [X])
end,
ping(N-1, Pong_PID, Ping).
Pong() ->
receive
```